

ELEC 450 Project 4

Max Kuhlman and Henry Prendergast

November 4, 2025

Problem Statement

For this project, we implemented kinematically constrained environments for a simple inverted pendulum problem and a car-like system in a simulated street environment. The problem definitions created included the ordinary differential equations defining the motion of each system, as well as projections for each system to be compatible with more planners. We then tested these systems with RRT and KPIECE planners, and implemented and benchmarked a Reachability-Guided RRT (RG-RRT) algorithm for the same problems.

Environments

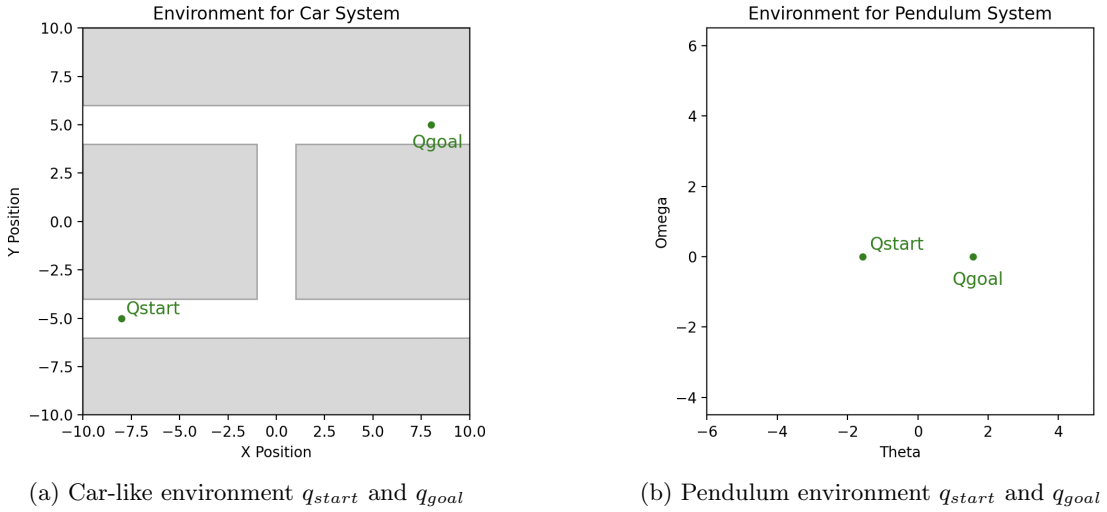


Figure 1: Visualization of the environments for the environments created for this project

For the pendulum environment (Figure 1b), the start state was the pendulum hanging down with a theta of $-\pi/2$ and an angular velocity $\omega = 0$, with the goal state being a theta of $\pi/2$ for an upright pendulum and an angular velocity of zero. For the car environment (1a), the start and goal states are at opposite corners of a simple street environment, similar to the one presented in the project 4 problem statement.

Robots and their Spaces

The pendulum is a simple pendulum controlled by a torque applied at the axis of rotation. The configuration space for the pendulum is 1-dimensional with the topology S^1 , the boundary of a circle in two dimensional space. The state space for the pendulum also includes the angular velocity, so has the topology $\mathbb{R}^1 \times S^1$. The control space for the pendulum is a single term, the torque, so is defined by $\mathbb{R}^1$.

The ‘car’ is modeled by a point in 2 dimensional space, though it has an angle which defines the available motion. This point can ‘rotate’ and accelerate forward or backward in the direction it is pointing. Thus, the configuration space of the car is three-dimensional with topology $SE(2)$. The state space of the car also includes the velocity of the car, so is $SE(2) \times \mathbb{R}^1$. The control space for the car includes the linear acceleration and angular velocity, so is two dimensional with topology $\mathbb{R}^2$.

Pendulum Solution Paths for Different Torques

To compare the paths for pendulum solutions, we visualized generated paths from each planner across the three torque options.

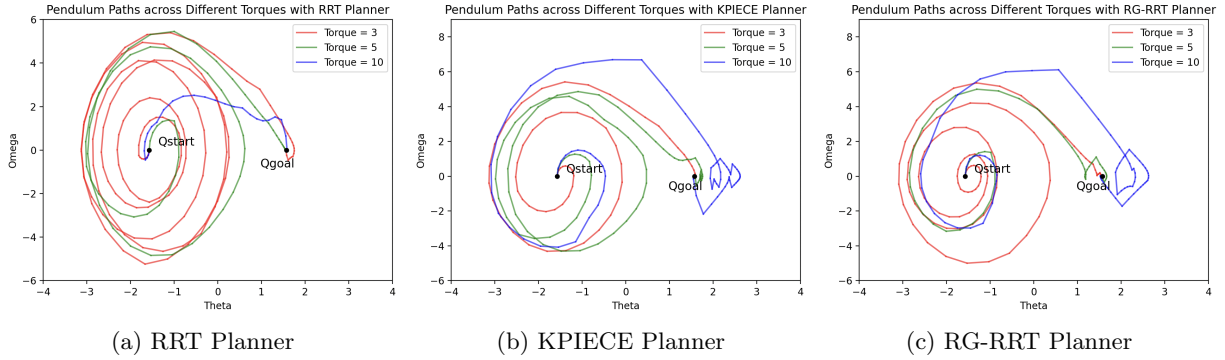


Figure 2: Visualization output comparing paths generated with different torque conditions.

We can see that no matter which planner is used, the solutions with higher torques require fewer swings to build up enough kinetic energy to lift the pendulum fully up. For the pendulum with a torque of 10 this is particularly noticeable, since these pendulums are powerful enough to rotate the pendulum upwards against gravity without building up substantial speed.

Benchmarking Data Summary

To compare the performance of the three planners, we used planner arena to represent the benchmarking data from testing each planner 20 times on each scenario.

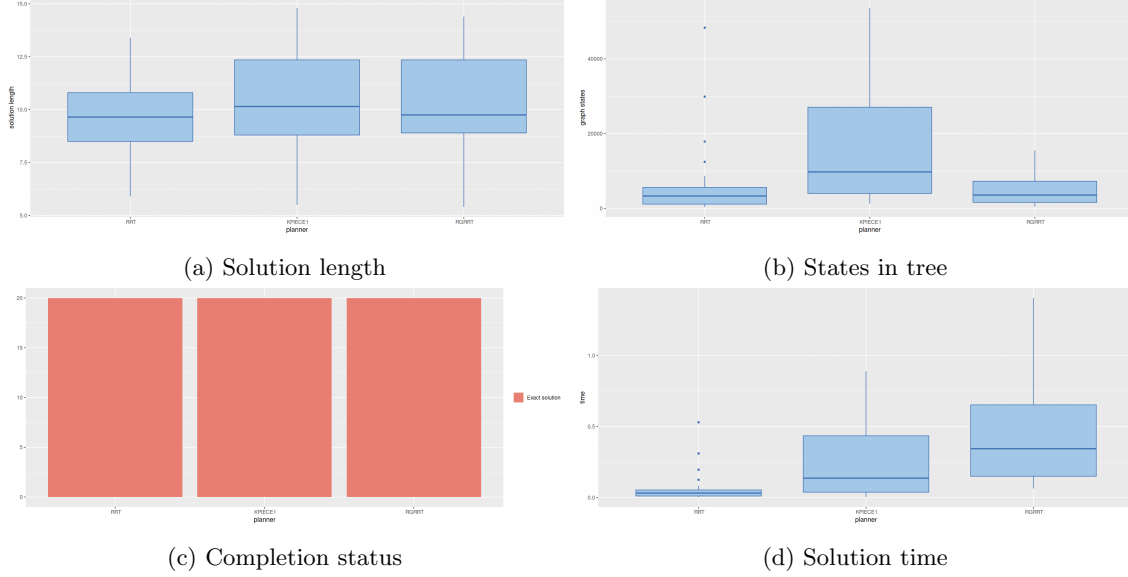


Figure 3: Benchmarking data from the pendulum scenario with a torque of 3.

When solving the pendulum scenario, all three planners were able to achieve an ‘exact’ solution for all 20 of their runs, with an average solution time 0.04 seconds for RRT, 0.13 seconds for KPIECE, and 0.35 seconds RG-RRT. All three planners had similar solution lengths around 10, with RRT edging the other two out to produce slightly shorter plans. Finally, for the size of the resulting tree, RRT is the smallest followed closely by RG-RRT, though RRT has several outliers with much larger sizes whereas RG-RRT doesn’t have any outliers.

Comparison of RRT and RG-RRT

To compare the RRT and RG-RRT algorithms, we used the planner data from the previous section and graphed a few example plans for a better qualitative, visual understanding of the paths generated.

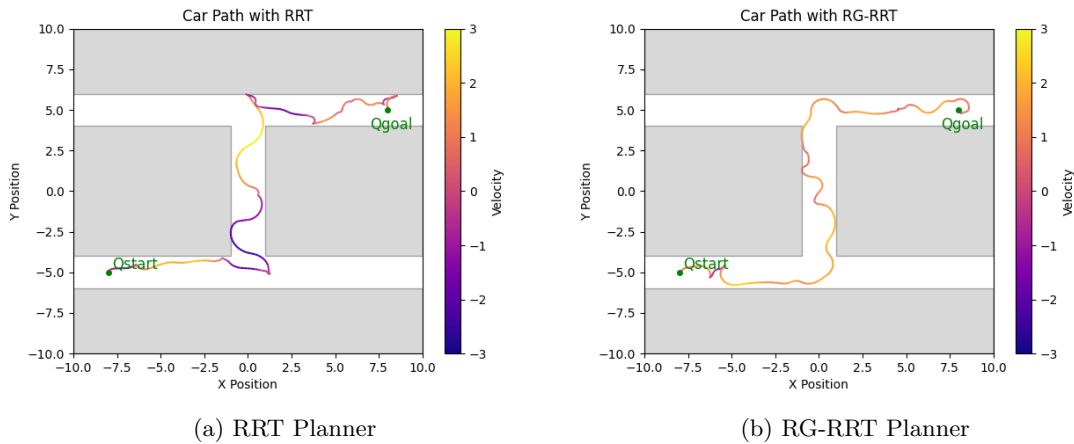


Figure 4: Visualization output of paths generated by RRT and RG-RRT algorithms in the car-like environment with point collision.

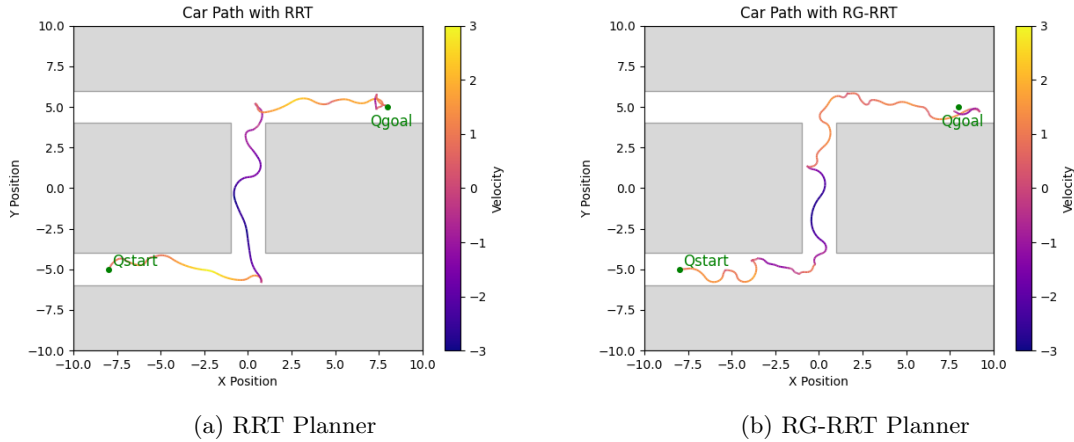


Figure 5: Visualization output of paths generated by RRT and RG-RRT algorithms in the car-like environment with square collision (side length = 0.5).

These paths were computed with a maximum allotted time of 60 seconds per planner to better capture the behavior of each planner. Both planners rely on random sampling to a significant degree, so their behavior is difficult to capture with a small number of paths. Generally, the RG-RRT succeeds in significantly reducing the size of the main state tree compared to RRT, but introduces a second set of reachable states that must be computed. The car problem was explored using both point collision and full square collision. For the point collision case depicted in Figure 4, the RRT path contains 99 control states out of a tree of 253450 total states. The RG-RRT path contains 75 control states out of a tree of 71234 states. This shows the significant reduction in tree states that RG-RRT provides; this may increase the speed of finding the nearest node in the tree to a new state. All is not as it seems, though: the RG-RRT tree also includes 166901 reachable states that had to be computed during solving, representing a similar total computational load to RRT. Figure 5 depicts the car problem with square collision. The RG-RRT path contains 96 different control states out of 47606 total states in the tree, not counting states that were computed as part of the reachability of each node. The RRT tree contains a total of 251344 states and the path contains 70 control states. The change in collision/validity checking is less important than the observation that RG-RRT is not always better than RRT; in this case, the path appears to be longer, slower, and less efficient. These paths were computed without smoothing, so it is possible that the RG-RRT path will benefit from smoothing much more than the RRT path, since many random states have been replaced with reachable states. These four paths were computed with a planning time of 60 seconds to exaggerate the differences between the two planners. Despite the extra-long planning time, none of these paths represent an exact solution, so comparing the planning time of the planners is not possible. For the pendulum case benchmarking shown in Figure 3, RRT and RG-RRT were able to successfully complete the motion planning problem. Planning for RG-RRT took a significantly longer time than RRT on average, although the planning time of RG-RRT was generally less variable (fewer outliers) than that of RRT. This may be because every valid state that is added to the RG-RRT tree must compute 11 more reachable states, and many of those will not be used. This increases the computational time required, since state propagation takes time. For RRT, if a state is valid, it is simply added to the tree.

Exercise Difficulty

1. Solving the motion planning problems for the pendulum and car with RRT

- This required a lot of work to set up, but we were able to refer to code from the previous projects to better understand the sequence of steps required to create a problem in OMPL. Took about 6 hours. The most difficult part was implementing the ODE propagator, but the OMPL documentation had a very useful tutorial for doing just that. 5/10.

2. Extending the motion planning problem to work with KPIECE
 - Once the problem and RRT implementation were in place, extending the code to use KPIECE was simple. Took about two hours. Projections for these systems are simple and intuitive, but they were probably the most difficult part. 3/10.
3. Implement RG-RRT
 - Successfully implementing RG-RRT was challenging and we encountered many issues. Took something like 14 hours. Adding reachable states to the tree was reasonably simple, but actually incorporating them into planning was difficult. 8/10.
4. Compare the planners using the Benchmark class
 - Using the Benchmark class in OMPL was simple and worked without many problems for our custom RG-RRT implementation. Took about one hour. The most difficult part was either just reading the documentation and figuring out the sequence required to successfully create and use the Benchmark, or figuring out that its constructor wanted a SimpleSetup, not a SimpleSetupPtr. 2/10.